

Kreacijski paterni

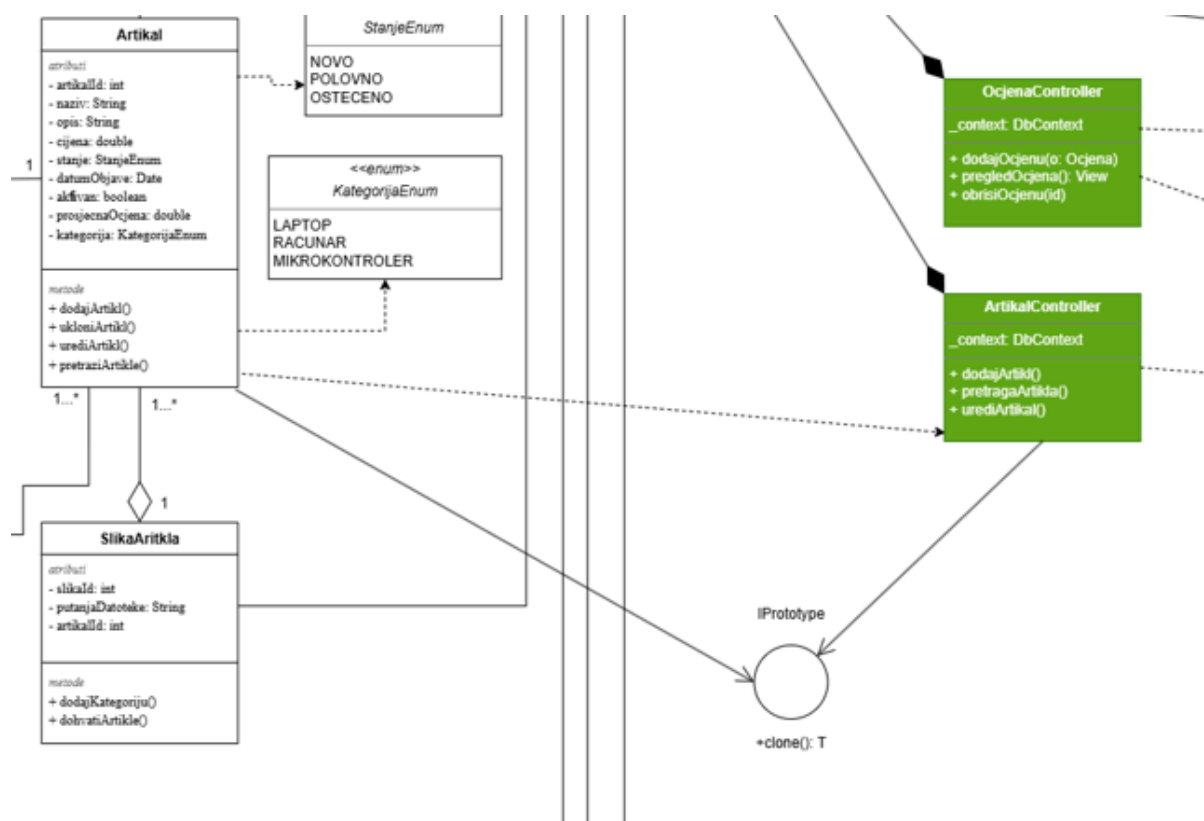
U sklopu razvoja sistema za kupovinu i iznajmljivanje artikala, identifikovana je potreba za uvođenjem kreacijskih dizajn paterna. Kreacijski paterni se bave procesom kreiranja objekata i pomažu u upravljanju složenošću instanciranja klasa. Odabrani su paterni koji se prirodno uklapaju u postojeću arhitekturu sistema i donose konkretne prednosti u pogledu fleksibilnosti, proširivosti i održavanja koda. Od pet standardnih kreacijskih paterna, za implementaciju su odabrani Prototype i Factory Method, dok preostala tri paterna – Singleton, Abstract Factory i Builder – nisu implementirana, ali je opisano gdje bi se mogli primijeniti u okviru sistema.

1. Prototype pattern

Osnovna funkcija Prototype paterna je da olakša kreiranje objekata koristeći postojeću instancu, koja se ponaša kao prototip. Novokreiranom objektu možemo promijeniti određene osobine po potrebi.

U našem sistemu za kupovinu i iznajmljivanje artikala, ovaj patern je primjenjen nad klasom **Artikal**. Prodavači često oglašavaju slične artikle koji dijele većinu atributa (naziv, kategorija, stanje, cijena). Umjesto ručnog ponovnog unosa svih podataka, prodavač može klonirati postojeći artikal kao predložak i izmijeniti samo specifične detalje za novi oglas. Na taj način se ubrzava proces kreiranja novih oglasa te se smanjuje mogućnost greške pri unosu.

Klasa **Artikal** implementira interfejs **IPrototype** koji sadrži metodu **clone()**. Ova metoda kreira i vraća kopiju postojećeg artikla. Kontroler **ArtikalController** dobija novu metodu **kloningArtikla()** koja poziva **clone()** nad odabranim artiklom i vraća novu instancu spremnu za uređivanje.

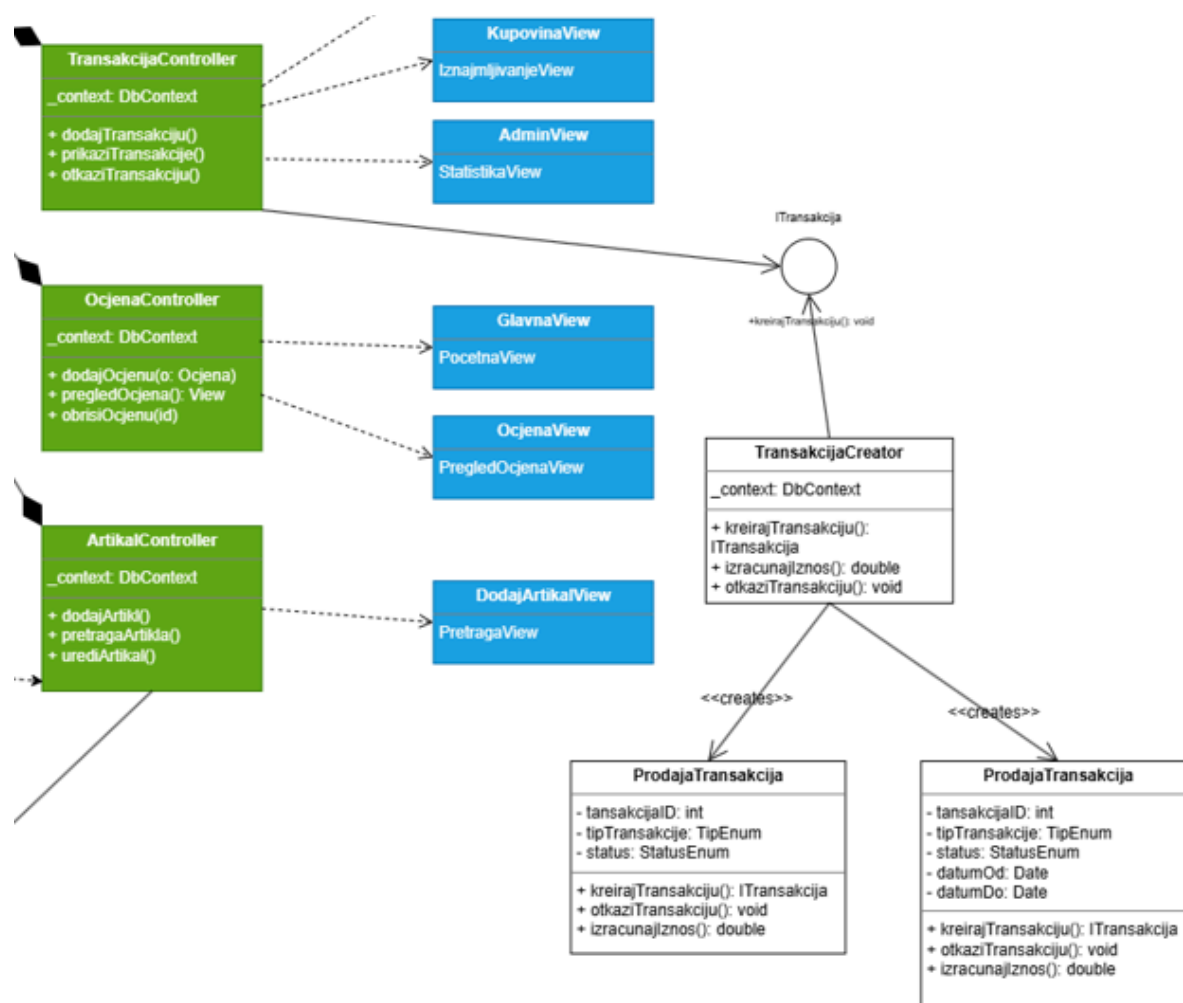


2. Factory Method pattern

Factory Method pattern omogućava kreiranje objekata na način da podklase odluče koju klasu instancirati. Ovo se radi preko interfejsa sa jednom metodom koju različite podklase mogu implementirati drugačije.

U našoj aplikaciji, ovaj pattern je primjenjen nad klasom **Transakcija**. Sistem podržava dva tipa transakcija definisana kroz enum **TipTransakcije**: **PRODAJA** i **IZNAJMLJIVANJE**. Svaki tip ima svoju logiku i specifične atribute — transakcija iznajmljivanja sadrži dodatne atribute **datumOd** i **datumDo**, dok transakcija prodaje ne zahtijeva te podatke.

Uveli smo interfejs **ITransakcija** sa metodom **kreirajTransakciju()** te klasu **TransakcijaCreator** koja sadrži factory metodu. Podklase **ProdajaTransakcija** i **IznajmljivanjeTransakcija** implementiraju ovaj interfejs svaka na svoj način. **TransakcijaController** koristi **TransakcijaCreator** za kreiranje odgovarajuće transakcije na osnovu korisnikovog odabira, čime se postiže odvajanje logike kreiranja od ostatka sistema.



3. Singleton pattern

Uloga Singleton paterna je da osigura da se klasa može instancirati samo jednom i da osigura globalni pristup kreiranoj instanci klase.

U našoj aplikaciji, Singleton bi se mogao primijeniti na klasu **DataContext**, koja predstavlja konekciju prema bazi podataka. Kreiranje višestrukih instanci konteksta baze podataka može dovesti do problema sa konzistencijom podataka i nepotrebnog trošenja resursa. Primjenom Singleton paterna osigurali bismo da cijela aplikacija dijeli jednu instancu **DataContext**-a, što bi poboljšalo efikasnost upravljanja konekcijama i smanjilo rizik od grešaka u pristupu podacima.

4. Abstract Factory pattern

Abstract Factory patern omogućava kreiranje više familija objekata sa mogućnošću dodavanja novih u budućnosti. Abstract Factory odvaja definiciju (klase) produkata od klijenta, zbog čega se familije produkata mogu jednostavno izmijenjivati ili ažurirati bez narušavanja strukture klijenta.

U kontekstu našeg sistema, Abstract Factory bi se mogao primijeniti za upravljanje različitim tipovima korisnika. Sistem trenutno poznaje dvije uloge — **Korisnik** i **AdminModerator** — koje imaju različite skupove dozvola i pristupa funkcionalnostima. Uvođenjem Abstract Factory paternu mogli bismo kreirati zasebne familije objekata za svaku ulogu, omogućavajući lakše dodavanje novih uloga u budućnosti (npr. premijum korisnik) bez mijenjanja postojeće strukture koda.

5. Builder pattern

Builder patern koristimo kada nam je potrebno primijeniti različite postupke za konstrukciju istog objekta.

U našoj aplikaciji, Builder patern bi se mogao primijeniti u procesu registracije korisnika. Klasa **Korisnik** sadrži više atributa (ime, prezime, email, šifra, uloga, datum registracije) čija se inicijalizacija može razlikovati ovisno o načinu registracije. Korisnik koji se prvi put registruje mogao bi odabrati između dva načina: automatskog generiranja korisničkog imena i lozinke putem **AutomatskiBuilder**-a, ili ručnog unosa tih podataka putem **RucniBuilder**-a. Oba buildera implementirali bi zajednički interfejs **Builder**, a klasa **KorisnikDirector** bi orkestrirala proces izgradnje bez ovisnosti o konkretnoj implementaciji.

